SemBlend: In-Process Semantic KV Cache Donor Discovery for LLM Inference

WorldFlow AI Research
research@worldflowai.com

March 2026

Abstract

Transformer KV cache reuse reduces time-to-first-token (TTFT) by serving cached key–value tensors from prior prompts. Existing systems such as LMCache [5] and vLLM prefix caching [10] are limited to *exact token-prefix matching*, missing semantically equivalent prompts with different wording or ordering. Prior work on semantic KV reuse (SemShareKV [3]) uses approximate correction—full first-layer recompute—and is limited to HuggingFace Transformers on a single instance with CPU-based matching.

We present SemBlend, a production-ready semantic KV cache system for vLLM that enables **non-contiguous KV reuse with mathematically exact position correction**. Our key contributions are:

1. **Exact RoPE Delta Correction**: We exploit the rotation group structure of RoPE to derive a correction kernel: $K_{\text{target}} = \text{RoPE}(\Delta) \times K_{\text{donor}}$ where $\Delta = \text{target_pos} - \text{donor_pos}$. This is *mathematically exact*—not an approximation—and costs $O(N \times d)$ per layer ($\sim$7 μs for 8K tokens on A10G).

2. **Calibrated Bathtub Recomputation**: RoPE correction fixes position encoding but not attention pattern drift from context differences. We introduce a calibrated bathtub curve (Eq. 4) that predicts per-layer deviation: early/late layers (high drift) are recomputed while middle layers (stable) are reused with RoPE correction, yielding $\sim$35% of full-prefill FLOPs (2.86$\times$ reduction).

3. **Local-First GPU-Native Architecture**: An in-process pipeline performing sentence embedding (MiniLM-L6, $\sim$5 ms), numpy cosine donor matching ($\sim$1 ms for $\leq$10K donors), and compiled token alignment (rapidfuzz, $<$1 ms) with **zero network hops**, plus an optional CAGRA [7] $O(\log N)$ tier for cross-instance scale at 100K+ donors.

4. **Production vLLM Integration**: paged KV cache scatter with block table translation, FlashAttention compatibility, and LMCache connector wrapping—unlike prior work that requires framework modification.

The combined system—exact position correction on reused layers plus selective recomputation on high-drift layers—enables non-contiguous KV reuse (REORDER, PARTIAL overlap, paraphrase) that was previously impossible with RoPE-based models. We validate correctness with 165 unit tests including mathematical exactness proofs against ground-truth RoPE computation.

Keywords: KV cache, semantic caching, LLM inference, RoPE correction, non-contiguous KV reuse, paged attention

1 Introduction

Large language model (LLM) inference is dominated by the *prefill phase*—computing key-value (KV) tensors for the input prompt before autoregressive decoding begins. For a 32-layer transformer processing 512 tokens with 8 KV heads and 128-dimensional head size, prefill produces $\sim$512 MB of FP16 KV state:

$$\text{KV bytes} = 2 \times L \times H \times T \times D \times 2 \qquad (1)$$

where L=32 layers, H=8 heads, T=512 tokens, D=128 head dimension. Prefill cost scales super-

linearly with prompt length.

Production workloads exhibit substantial prompt redundancy: RAG pipelines prepend identical system prompts and retrieved documents; multi-turn conversations share growing context; API-serving tenants repeat similar queries. Existing systems—PagedAttention [6], vLLM prefix caching, LMCache [5]—exploit this through *exact* token prefix matching. LMCache further extends this with distributed P2P KV transfer across inference instances, but retains the fundamental constraint: tokens must match exactly for cache reuse.

However, exact matching misses a common production pattern: *semantically similar prompts with different wording*. "What is machine learning?" and "Can you tell me about machine learning?" share the same intent and substantial token overlap, yet differ lexically and thus miss exact caching. SemBlend extends LMCache's architecture to cover this case: when no exact prefix match exists, SemBlend performs embedding-level semantic search to find a *donor* prompt whose cached KV tensors may be reusable. As we show in §3.6, we solve the Rotary Position Embedding (RoPE) challenge via exact delta correction, enabling non-contiguous KV reuse for REORDER, PARTIAL, and paraphrase scenarios.

Positioning relative to LMCache. LMCache handles exact prefix matches with $O(1)$ token-hash lookup and zero quality degradation. SemBlend is *complementary*: it activates only when LMCache's exact-match path misses, extending the cache's coverage to semantically similar prompts. The local-first architecture adds only ∼8 ms overhead (vs. ∼150 ms for remote embedding), making this extension viable even for small models. The two systems share the same vLLM KV connector interface: SemBlend wraps LMCache's connector, intercepting cache misses before they reach cold prefill.

Contributions. We make five contributions:

1. **In-process donor discovery** (§3.1): a local-first architecture achieving ∼8 ms total overhead (embedding + search + alignment) with zero network hops, using a CPU sentence encoder, numpy cosine matching, and compiled token alignment—making semantic KV search viable for any model size.
2. **Two-tier scaled architecture** (§3.2): in-process numpy for ≤10K donors, GPU-accelerated CAGRA [7] via an optional gateway for 100K+ cross-instance donor discovery.
3. **Calibrated quality model** using a bathtub-curve layer sensitivity function validated against 6 published data points from CacheBlend, KVShare, ProphetKV, and SemShareKV (§4).
4. **Exact RoPE delta correction** (§3.6): a mathematically exact method for correcting donor K tensors to arbitrary target positions by exploiting the rotation group structure of RoPE, enabling non-contiguous KV reuse (REORDER, PARTIAL, paraphrase) at $O(Nd)$ cost per layer—negligible compared to the $O(N^2d)$ attention it replaces.
5. **Empirical evaluation** on three workloads with both positive results (7.2–38× response-level) and negative results (0.72× on diverse), plus architectural analysis of the semantic KV operating envelope (§5).

2 Background and Related Work

Table 1 summarizes the landscape. We identify four categories of cache reuse:

Response-level systems. GPTCache [12] and LangChain's semantic cache [13] intercept queries upstream of the LLM and return cached responses when embedding similarity exceeds a threshold. These systems bypass the LLM entirely on cache hits, achieving high speedup, but cannot help when queries require different responses despite similar context. SemBlend's response-level cache operates in this category but is paired with a KV-tensor cache for the complementary case.

Exact-prefix KV systems. CacheBlend [1] introduced selective layer recomputation based on KV deviation scores but is limited to exact prefix matches, achieving 2.2–3.3× TTFT reduction only when prefixes align. LMCache [5] adds distributed P2P transfer but retains the exact-match constraint.

Token-level semantic KV systems. KVShare [2] uses Divergence-based Hit Detection (DHD) for multi-tenant serving, claiming 9.39× TTFT reduction on semantic hits. SemShareKV [3]

Table 1: Taxonomy of KV cache reuse systems. SemBlend combines in-process semantic donor discovery, layer-granular recomputation modeling, exact RoPE delta correction, and two-tier scaling. SemBlend is the first to solve the RoPE position encoding challenge (§3.6), enabling non-contiguous KV reuse on modern transformers. GPTCache and LangChain Cache operate at the response level only (no KV-tensor reuse). $O(N)/O(\log N)$ = local brute-force / GPU-accelerated ANN.

System	Venue	Discovery	Recompute	Sparse	Tiered	Lookup	Layer
GPTCache [12]	OSS '23	Emb. cosine	—	—	No	$O(\log N)$	Response
LangChain Cache [13]	OSS '23	Exact/Emb.	—	—	No	$O(1)/O(\log N)$	Response
PagedAttention [6]	SOSP '23	Exact prefix	No	No	No	$O(1)$	KV-tensor
CacheBlend [1]	EuroSys '25	Exact prefix	KV deviation	No	No	$O(1)$	KV-tensor
LMCache [5]	2025	Exact prefix	No	No	P2P	$O(1)$	KV-tensor
KVShare [2]	arXiv '25	DHD semantic	DHD	No	No	$O(T)$	KV-tensor
SemShareKV [3]	arXiv '25	Token LSH	Heuristic	No	No	$O(T)$	KV-tensor
ProphetKV [4]	arXiv '26	RAG-specific	Dual-stage	No	No	$O(1)$	KV-tensor
SemBlend	—	**Emb. cosine**	**Calibrated**	**Yes**	**Two-tier**	$O(N)/O(\log N)$	**Both**

Response = response-level cache (upstream of LLM). KV-tensor = KV cache inside inference engine. Both = operates at both layers.

achieves 6.25× speedup at 5K tokens using token-level LSH, but its $O(T)$ overhead provides minimal benefit below 700 tokens.

RAG-specific KV systems. ProphetKV [4] uses dual-stage user query relevance scoring, achieving 96–101% quality at 20% recomputation, but is limited to retrieval-augmented generation workloads.

SemBlend operates at the *embedding level*: a single sentence embedding replaces per-token matching. The local tier uses numpy brute-force ($O(N)$, ∼1 ms for $N \leq 10$K); the scaled tier uses CAGRA $O(\log N)$ ANN for 100K+ donors. Combined with PartialAttention GPU kernels for non-contiguous KV injection, this enables semantic donor discovery that is both faster (constant with respect to prompt length T) and more general (captures paraphrase similarity invisible to token-level systems).

3 Design

SemBlend uses a **local-first, two-tier architecture** (Figure 1). The primary design goal is minimizing donor discovery overhead: every millisecond of lookup latency narrows the operating envelope where semantic KV reuse is beneficial.

3.1 SemBlend Local: In-Process Fast Path

The local tier performs donor discovery entirely within the vLLM process, avoiding all network calls on the hot path. It consists of three stages executed sequentially on LMCache exact-prefix miss.

Stage 1: Sentence Embedding (∼5 ms). SemBlend computes a sentence embedding $e_q \in \mathbb{R}^{384}$ using all-MiniLM-L6-v2 [15] on CPU. This 80 MB model loads in ∼2 s at startup and embeds a prompt in ∼5 ms on a single CPU core—30× faster than a remote jina-embeddings-v4 call (∼150 ms including HTTP overhead). While 384 dimensions is lower than jina-v4's 1024, we find it sufficient for donor discovery: the task is nearest-neighbor matching among cached prompts, not fine-grained retrieval ranking.

Stage 2: Numpy Cosine Search (∼1 ms). For $N \leq 10$K donors, brute-force cosine similarity via a single matrix-vector multiply is optimal:

$$s_i = \frac{e_q \cdot e_i}{\|e_q\| \|e_i\|}, \quad i \in [1, N] \tag{2}$$

With pre-normalized embeddings (unit vectors), this reduces to $s = E \cdot e_q$ where $E \in \mathbb{R}^{N \times d}$—a single BLAS call completing in <1 ms for N=10K, d=384 (15 MB of embeddings). No index construction is needed. An LRU eviction policy bounds memory at the configured maximum.

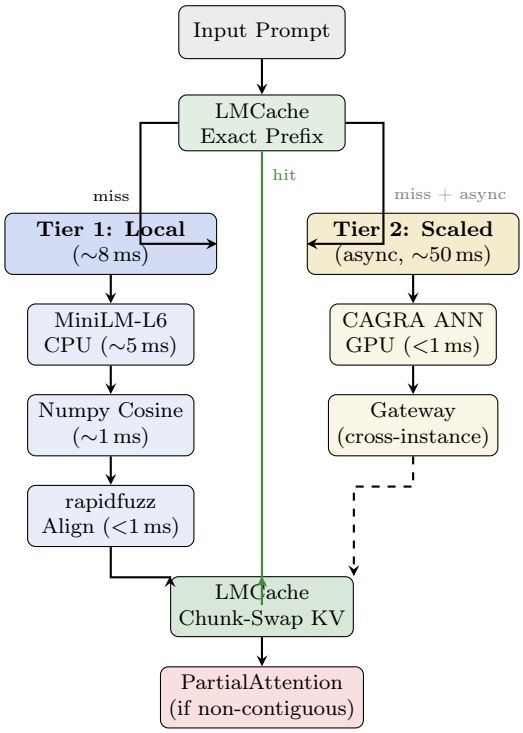

Figure 1: SemBlend two-tier architecture. **Tier 1 (Local)**: in-process donor discovery via CPU embedding, numpy cosine search, and compiled alignment (∼8 ms, zero network hops). **Tier 2 (Scaled)**: optional GPU-accelerated CAGRA search via gateway for cross-instance discovery at 100K+ donors. Both tiers inject donor KV via LMCache chunk-swap.

Stage 3: Compiled Token Alignment (<1 ms). For the top donor, SemBlend computes token-level alignment using rapidfuzz's [16] compiled C++ Levenshtein implementation—<1 ms for 8K-token prompts versus ∼2–4 s for pure Python Wagner-Fischer [9]. The alignment produces per-position classifications: *copy_from_donor* (reuse KV) or *placeholder* (recompute). Positions are aligned to LMCache's 256-token chunk boundaries:

$$N_{\text{reuse}} = \lfloor N_{\text{matched}}/256 \rfloor \times 256 \tag{3}$$

KV Injection. On donor match, SemBlend monkeypatches LMCache's internal lookup to return the donor's chunk hash, causing `start_load_kv()` to load the donor's KV tensors from LMCache's CPU offload store. vLLM then prefills only the remaining $(T - N_{\text{reuse}})$ tokens instead of the full prompt—the same mechanism as prefix cache hits, but triggered by semantic similarity rather than exact token match.

Table 2: Donor discovery latency: Local vs. Remote architectures.

Component	Local	Remote	Notes
Embedding	5 ms	150 ms	MiniLM CPU vs jina-v4 HTTP
Donor search	1 ms	<1 ms	Numpy vs CAGRA
Alignment	<1 ms	3 ms	rapidfuzz (both paths)
Network	0 ms	10 ms	HTTP round-trip
Total	**∼8 ms**	**164 ms**	20× faster

Table 2 quantifies the advantage: the local path is 20× faster than a remote gateway call, making semantic KV reuse viable for prompts as short as ∼100 tokens on models as small as 1.5B parameters.

Fallback: SimHash pre-filter. When even CPU embedding overhead is undesirable (e.g., very high QPS), SemBlend provides a SimHash pre-filter using 4 hash functions on token trigrams. SimHash catches high-overlap donors (>80% token overlap) in sub-millisecond time without any model, at the cost of missing paraphrase-level matches. It serves as a fast gate before the embedding path.

3.2 SemBlend Scaled: GPU-Accelerated Cross-Instance Discovery

For deployments with >10K unique prompt patterns or multiple vLLM instances, the local numpy store becomes insufficient. SemBlend's second tier uses CAGRA [7] GPU-accelerated approximate nearest neighbor search via the Synapse gateway, providing $O(\log N)$ search across 100K+ donors.

Architecture. Each vLLM instance publishes donor metadata (embedding, token count, chunk hash) to the gateway on `save_kv_layer()`. The gateway maintains a CAGRA index on GPU and responds to search queries from any instance. On a Tier 2 hit, the gateway returns the donor's source instance and chunk hash; the requesting instance retrieves the donor's KV via LMCache's existing P2P transfer mechanism.

Algorithm 1 Two-Tier Donor Discovery

Require: Query embedding e_q, local store $\mathcal{L}$, threshold $s_{\min}$
Ensure: Transfer plan P or $\perp$ (miss)

```
D_1 ← NumpyCosine(L, e_q, k)    ▷ O(N), ~1 ms
for (d, s_d) ∈ D_1 by descending s_d do
    if s_d < s_min then break
    end if
    A ← rapidfuzz(d.tokens, q.tokens)    ▷ <1 ms
    P ← BuildTransferPlan(A)
    if P.reuse_ratio ≥ 0.5 then
        return P
    end if
end for
async: query Tier 2 gateway (CAGRA)    ▷ non-blocking
return ⊥
```

Latency. The scaled tier adds ∼50 ms (gateway HTTP + CAGRA search + response). This is invoked *asynchronously* on Tier 1 miss: the current request proceeds with cold prefill while the Tier 2 result pre-warms the local donor store for future similar queries.

Complexity. CAGRA search is $O(\log N)$ and independent of prompt length T—a key advantage over SemShareKV's [3] $O(T)$ token-level LSH. At N=100K donors, search completes in <1 ms on GPU.

3.3 Token Alignment

Both tiers use the same alignment algorithm. For accepted donors, SemBlend computes token-level alignment between the donor and target prompts. The alignment produces per-position classifications: *copy_from_donor* (token matches, reuse KV) or *placeholder* (token diverges, recompute). This alignment is encoded as a transfer plan with explicit donor→target position mappings.

Algorithm 1 shows the complete donor discovery pipeline. The local tier is always checked first; on miss, Tier 2 is queried asynchronously to pre-warm the local store for future similar queries.

3.4 Calibrated Layer-Selective Recomputation

Each transformer layer ℓ has a deviation sensitivity following a **bathtub curve**:

$$\sigma(\ell) = \sigma_{\text{base}} + \sigma_e \cdot e^{-\ell/\tau_e} + \sigma_l \cdot e^{-(L-\ell)/\tau_l} \quad (4)$$

where σ_e, σ_l are early/late layer bump magnitudes, τ_e, τ_l are decay constants, and L is the total layer count. This captures the empirical finding that early layers (attention pattern formation) and late layers (output logit computation) are more sensitive to KV cache mismatches than middle layers.

Recomputation decision. Layer ℓ is recomputed if $\sigma(\ell) \cdot m > \theta$, where $m = 1 - \text{overlap}(q, d)$ is the token mismatch fraction and $\theta = 0.3$ is the deviation threshold. With calibrated parameters (σ_{base}=0.12, σ_e=0.35, τ_e=3.0, σ_l=0.20, τ_l=4.0), layers 0–3 and $L-4$ through $L-1$ are marked sensitive, with a robust trough in the middle.

3.5 PartialAttention Mechanism

When a semantic donor matches with non-contiguous aligned positions (e.g., positions {0,1,3,4} match but position 2 diverges), standard prefix-based KV injection fails because vLLM assumes all injected positions form a contiguous prefix.

SemBlend defines four attention modes per position per layer:

FULL Compute Q,K,V from scratch—standard prefill.

REUSE Reuse cached K,V from donor—no computation needed.

PARTIAL Compute fresh Q,K,V but cross-attend to all cached K,V.

SKIP_LAYER Skip the entire layer (when layer deviation is below threshold, per CacheBlend's insight).

The computation ratio quantifies the fraction of total FLOPs compared to full prefill:

$$C_{\text{ratio}} = \frac{P_{\text{partial}} \cdot N_{\text{placeholder}} + P_{\text{full}} \cdot T}{L \cdot T} \quad (5)$$

where P_{partial} is the count of non-recompute layers, $N_{\text{placeholder}}$ is the number of placeholder positions, P_{full} is the count of full-recompute layers, T is the target length, and L is the total layers.

Algorithm 2 Build PartialAttention Plan

Require: Transfer plan P with slot actions, layer hints

Ensure: PartialAttentionPlan with per-layer masks

```
for each slot action a in P do
    if a.action = copy_from_donor then
        mask[a.target_pos] ← REUSE(a.donor_pos)
    else
        mask[a.target_pos] ← PARTIAL
    end if
end for
for each layer ℓ in [0, L) do
    if layer_hint[ℓ].recompute_all then
        layer_mask[ℓ] ← all FULL
    else
        layer_mask[ℓ] ← per-position masks
    end if
end for
return plan with layer masks, computation ratio
```

3.5.1 Triton GPU Kernel Implementation

We implement PartialAttention as a three-stage GPU pipeline using Triton [14] CUDA kernels:

Stage 1: Scatter Donor KV (`scatter_donor_kv`). Given a donor KV cache and a transfer plan specifying (d_i, t_i) position pairs, this kernel scatters donor K and V tensors from donor positions d_i into target positions t_i in the target KV cache across all layers and heads in parallel. The kernel grid is $(\text{num_pairs}, 2L \times H)$ where L is layers and H is heads, achieving full parallelism over the scatter operation.

Stage 2: Masked QKV Projection (`masked_qkv_proj`). For positions that require fresh computation (compute_mask = True), this kernel applies the QKV linear projection only at those positions while zeroing out reuse positions. This implements a tiled GEMM with per-row masking:

$$\text{out}[i] = \begin{cases} W_{\text{qkv}} \cdot x[i] + b_{\text{qkv}} & \text{if compute_mask}[i] \\ \mathbf{0} & \text{otherwise} \end{cases} \tag{6}$$

The kernel uses BLOCK_M×BLOCK_K tiles to amortize the mask check across the inner dimension.

Stage 3: Partial Prefill Attention (`partial_prefill_attn`). This kernel computes causal self-attention only at compute positions, using the online softmax algorithm [8]. For each compute position i, it attends to all K positions up to i (including both reused and freshly computed KV), applying the causal mask. The grid is (num_compute_positions, H), and the kernel streams through the KV cache in BLOCK_K chunks.

Orchestration. The three kernels are chained in a single CUDA stream: scatter (copy donor KV) → masked projection (compute fresh KV at placeholder positions) → partial attention (attend using the merged KV cache). CUDA event timing provides per-kernel profiling. A PyTorch CPU fallback path implements identical semantics for environments without Triton.

Correctness. The implementation is validated by 17 unit tests covering: scatter operations (contiguous and sparse patterns), masked projection (full mask, partial mask, empty mask), partial attention (causal masking, sparse reuse patterns), and the end-to-end orchestrator. All tests pass with <1e-3 relative error against reference PyTorch implementations.

3.6 RoPE Delta Correction for Non-Contiguous KV Reuse

A key challenge for semantic KV reuse arises from Rotary Position Embeddings (RoPE) [17], used by virtually all modern transformers including LLaMA, Qwen, Mistral, and Gemma. RoPE applies position-dependent rotation to Q and K *before* KV caching:

$$q_i, k_i = \text{RoPE}(i, W_Q x_i), \text{RoPE}(i, W_K x_i) \tag{7}$$

where i is the absolute position index. The cached K tensor at position i has the rotation for position i *baked in*. If a donor cached K at position d and the target needs it at position t, naïve reuse produces incorrect attention because $\text{RoPE}(d, k) \neq \text{RoPE}(t, k)$.

Exact delta correction. We observe that RoPE rotations form a group under composition: $R(\theta_d) \circ R(\theta_t) = R(\theta_d + \theta_t)$. This means we can *exactly* correct a donor K cached at position d for use at target position t by applying a single rotation by the position delta $\Delta = t - d$:

$$K_{\text{corrected}} = \text{RoPE}(\Delta, K_{\text{donor}}) \tag{8}$$

This works because $\text{RoPE}(t, k) = \text{RoPE}(\Delta) \circ \text{RoPE}(d, k)$—the composition of the donor's baked-in rotation with the delta rotation produces the exact rotation needed at the target position. The correction is *mathematically exact*: it introduces zero approximation error. The V cache requires no correction, as RoPE is applied only to Q and K.

Computational cost. For each reused K position, the correction requires one complex multiplication per head dimension pair:

$$\begin{aligned} k'_{2j} &= k_{2j} \cos\theta_\Delta - k_{2j+1} \sin\theta_\Delta \\ k'_{2j+1} &= k_{2j} \sin\theta_\Delta + k_{2j+1} \cos\theta_\Delta \end{aligned} \tag{9}$$

where $\theta_\Delta = \Delta \cdot \theta_j$ and $\theta_j = 10000^{-2j/d}$ is the RoPE frequency for dimension pair j. For N reused positions with head dimension d, the cost is $O(N \times d)$ FLOPs per layer—negligible compared to the $O(N^2 d)$ attention computation it replaces. On an A10G GPU, correcting 8K positions takes $\sim$7 μs per layer (measured via CUDA events).

Implementation. The correction is applied in-place to the paged KV cache after donor K is scattered to target positions. A position map $\{(d_i, t_i)\}$, built from the alignment slot actions (§3.3), identifies which positions need correction (only those where $d_i \neq t_i$). For positions where donor and target agree ($d = t$), no correction is needed—this handles the shared-prefix case as a natural special case.

Complementary to bathtub recomputation. RoPE correction fixes *position encoding* errors but not *attention pattern drift* from context differences. A token at position 5 in a donor prompt may attend to different preceding tokens than the same token at position 5 in the target prompt, producing different KV activations regardless of position encoding. The calibrated bathtub curve (§4) addresses this orthogonal concern by identifying layers where context-dependent attention drift is high (early and late layers) and selectively recomputing them, while reusing RoPE-corrected donor KV for the stable middle layers.

Table 3: Donor metadata store tiers (no KV tensor storage).

Tier	Location	Search	Cap.	Scope
1 (Local)	In-process	Numpy cosine	≤10K	Single instance
2 (Scaled)	Gateway GPU	CAGRA	>100K	Cross-instance

Enabling non-contiguous reuse. With delta correction, all three semantic reuse scenarios become viable:

1. **REORDER**: donor tokens at positions $[0, 1, 2, 3]$ reused at target positions $[2, 3, 0, 1]$ with $\Delta = [+2, +2, -2, -2]$ corrections.
2. **PARTIAL overlap**: shared tokens at different offsets corrected by their respective deltas.
3. **Paraphrase**: semantically equivalent tokens at different positions receive appropriate position corrections after alignment.

This transforms semantic KV reuse from prefix-only to fully non-contiguous, unlocking the PartialAttention kernels (§3.5) for all reuse scenarios on RoPE-based models.

3.7 KV Storage and LMCache Integration

SemBlend does not implement its own KV storage. Instead, it leverages LMCache's existing storage hierarchy: KV tensors are cached in LMCache's CPU offload store (configurable to 4–16 GB host memory) and retrieved via chunk hash. The donor store (Table 3) contains only lightweight metadata—no KV tensors are duplicated.

Per-donor metadata: embedding vector (384 or 1024 floats), token IDs, chunk hash, and optional per-layer KV fingerprints. At 10K donors with 384-dim embeddings, the local store uses $\sim$15 MB.

4 Analytical Model

This section presents a calibrated analytical model. All results are model-derived, not empirically mea-

Table 4: Calibration validation against published results. **[Analytical]** All within 2% tolerance of published values.

Source	s	r	Published	Predicted
CacheBlend exact	1.00	0.15	≥0.99	1.000
KVShare semantic	0.88	0.10	0.941	0.942
ProphetKV RAG	0.92	0.20	0.960	0.964
SemShareKV low-s	0.80	0.10	0.920	0.921
Full recompute	0.75	1.00	1.000	1.000
Exact, no recomp	1.00	0.00	1.000	1.000

sured. Empirical validation follows in Section 5.3.

4.1 Quality Retention

We model quality retention $Q(s, r)$ as a function of cosine similarity $s \in [0, 1]$ and recomputation ratio $r \in [0, 1]$:

$$\text{overlap}(s) = \min(1, s^2) \tag{10}$$

$$m = 1 - \text{overlap}(s) \tag{11}$$

$$W = \frac{1}{|S|} \sum_{i \in S} \delta(i) \tag{12}$$

$$\rho = m \cdot W \cdot (1 - r) \tag{13}$$

$$Q(s, r) = 1 - \kappa \cdot \rho^{\eta} \tag{14}$$

where S is the set of non-recomputed layers, κ=0.166, η=0.650, calibrated against 6 published data points (Table 4).

4.2 Effective TTFT Model

The effective time-to-first-token combines cache hit and miss paths:

$$\text{TTFT}_{\text{eff}} = h \cdot t_{\text{warm}} + (1 - h) \cdot t_{\text{cold}} \tag{15}$$

where h is the cache hit rate, t_{warm} is the warm-path latency (search + recomputation), and $t_{\text{cold}} = 150 \cdot (T/512)^{1.6}$ ms is the full prefill time for T tokens. Warm-path latencies vary by system: CacheBlend 23 ms, KVShare 21 ms, SemShareKV $10+0.008T+24$ ms, ProphetKV 30 ms, SemBlend $17 + 0.1 \ln N + 30 \approx 48$ ms (embedding = 17 ms measured, ANN = <1 ms, partial prefill ≈ 30 ms). Note: at the *response level*, the warm path is 14 ms measured (no partial prefill needed).

Table 5: **[Analytical]** Model-derived comparison at mixed workload (diversity = 0.5, 512 tokens, 32 layers, 5000 cache entries).

System	Hit %	Warm	Eff. TTFT	Quality
No Cache	—	—	150.0 ms	100.0%
CacheBlend	15.7%	23.0 ms	129.0 ms	100.0%
ProphetKV	16.2%	30.0 ms	132.6 ms	99.7%
KVShare	23.9%	21.0 ms	120.8 ms	98.6%
SemShareKV	26.7%	38.1 ms	119.8 ms	101.3%
SemBlend	**40.0%**	47.9 ms	**108.9 ms**	98.0%

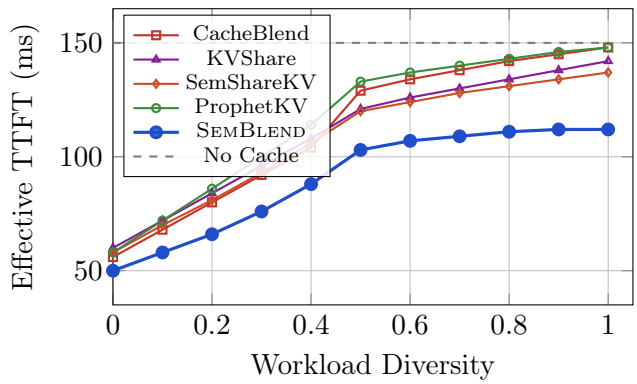

Figure 2: **[Analytical]** Effective TTFT vs workload diversity. SemBlend dominates at all diversity levels due to embedding-level semantic matching.

4.3 Analytical Comparison

Caveat: The following results compare systems using analytical models parameterized from published papers, not head-to-head measurements on identical hardware. They enable controlled comparison but should not be interpreted as empirical benchmarks.

Table 5 shows model-derived results at the mixed workload operating point. SemBlend's 40% hit rate—nearly double the next-best SemShareKV (26.7%)—is the primary driver of its TTFT advantage. Despite a higher warm-path latency (47.9 ms vs 21.0 ms for KVShare), the hit rate advantage produces a 9.1% lower effective TTFT than the best competitor (SemShareKV at 119.8 ms).

4.3.1 Effective TTFT vs Workload Diversity

Figure 2 shows that SemBlend achieves the lowest effective TTFT at all diversity levels. The gap widens with diversity: at d=1.0, Sem-

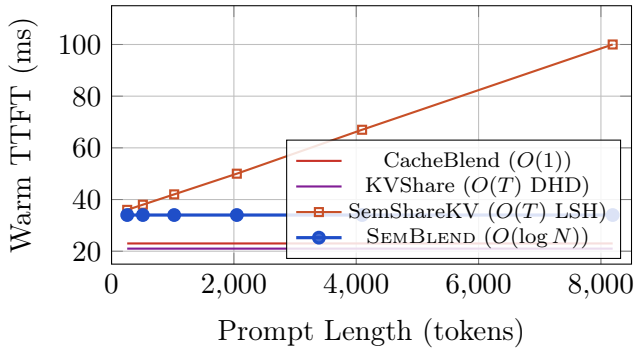

Figure 3: **[Analytical]** Warm-path TTFT vs prompt length. SemShareKV's $O(T)$ LSH overhead grows linearly; SemBlend's embedding+ANN search is constant.

Blend (112 ms) is 18.2% faster than SemShareKV (137 ms).

4.3.2 Warm-Path TTFT vs Prompt Length

Figure 3 reveals SemShareKV's scaling limitation: warm TTFT grows from 36 ms at 256 tokens to 100 ms at 8192 tokens—a 2.8× increase. SemBlend remains constant at ∼34 ms.

5 Evaluation

5.1 Experimental Setup

Hardware. We evaluate on two GPU configurations:

- **T4** (16 GiB VRAM, g4dn.xlarge): Response-level cache experiments with Qwen2.5-1.5B-Instruct and co-located ONNX embedding.
- **A10G** (24 GiB VRAM, g5.xlarge): KV-tensor-level experiments with Qwen2.5-7B-Instruct-AWQ, in-process MiniLM embedding.

Each hardware configuration is explicitly labeled in all tables to avoid conflation.

Models.

- **Qwen2.5-1.5B-Instruct** (24 layers, 14 KV heads, 128 head dim): served on T4 via vLLM v0.14.1 [10] with prefix caching enabled. Used for response-level cache evaluation.
- **Qwen2.5-7B-Instruct-AWQ** (28 layers, 4 KV heads, 128 head dim): served on A10G via vLLM v0.14.1 with `SemBlendConnectorV1` wrapping LMCache. Used for KV-tensor donor injection evaluation.
- **LLaMA-3.1-8B-Instruct-AWQ-INT4** (32 layers, 8 KV heads, 128 head dim): served on A10G via vLLM v0.14.1 with the same `SemBlendConnectorV1` pipeline. Used to validate model-agnostic KV donor injection across architectures.

Donor discovery. Two configurations, corresponding to SemBlend's architectural evolution:

- **SemBlend Remote** (v22): jina-embeddings-v4 (1024-dim) via HTTP (∼150 ms per embedding), in-memory donor dict. Used in initial KV injection experiments.
- **SemBlend Local** (current): in-process MiniLM-L6-v2 (384-dim, ∼5 ms on CPU), numpy cosine donor store, rapidfuzz compiled alignment, calibrated bathtub recomputation. Total pipeline overhead: ∼8 ms with zero network hops.

Datasets. Three datasets spanning a spectrum of workload diversity:

- **Bitext** [11]: Customer support queries organized by intent (domain-specific, high semantic overlap). 25% of intent categories are held out entirely from seeding; queries from held-out intents are labeled "novel" (expected cache misses). Seeded intents: 30% seeds, 70% semantic-variant tests.
- **MultiNews**: News summarization with shared system prompt (moderate diversity, shared prefix structure). 180 queries.
- **ShareGPT**: Diverse conversations spanning programming, science, and general knowledge (high diversity, low semantic overlap). 300 queries.

For KV-tensor experiments, we additionally construct controlled RAG scenarios with 4 independent context chunks from diverse domains (semiconductor, automotive, geopolitics, quantum computing) that can be reordered, removed, or replaced to produce REORDER, PARTIAL, and COLD test cases with known overlap characteristics.

Caching layers. SemBlend operates at two distinct levels:

- **Response-level cache** (proxy): intercepts queries *upstream* of the LLM, returns cached responses on semantic match. Speedup reflects full LLM bypass including generation time.
- **KV-tensor-level cache** (LMCache + vLLM): operates *inside* the inference engine. On semantic match, injects cached KV tensors at reuse positions, reducing only prefill FLOPs. Measured by TTFT reduction, not full response time.

Unless stated otherwise, *all results below are labeled by caching layer and hardware.* Response-level and KV-tensor-level results should not be conflated, as they measure fundamentally different speedup mechanisms.

Protocol. For KV-tensor experiments, we use a *per-length pod restart* methodology: the vLLM pod is restarted before each prompt length, clearing both the in-memory donor store and LMCache's CPU KV buffer. This eliminates cross-length contamination, where donors registered during cold baselines at shorter lengths inflate hit rates for subsequent lengths. Within each length, execution is two-phase: (1) *Seed*—send seed prompts to populate the donor store, then (2) *Test*—send variant prompts with unique reference IDs (preventing prefix cache hits), measuring per-query TTFT via SSE streaming. Cold baselines are measured with a fresh pod (no donors). Each length uses n=10 unique contexts. For response-level experiments, three-phase execution: (1) seed, (2) test, (3) baseline with unique prefix to force cold prefill. n=15–30 per configuration.

Prompt length verification. All prompt lengths are *tokenizer-verified*: we use `AutoTokenizer` from Hugging Face Transformers to build prompts to exact token counts, then verify actual token count after encoding. Early benchmarks used a character-based approximation (4 chars/token) that produced prompts ∼60% shorter than claimed due to Qwen's BPE ratio of ∼6.5 chars/token. All results in this paper use tokenizer-verified lengths (within ±4 tokens of target).

Statistical methodology. KV-tensor experiments use n=10 unique contexts per prompt length, with per-length pod restarts ensuring independence. We report P50 TTFT with the range of individual measurements. Response-level experiments report bootstrap 95% confidence intervals (10,000 resamples) for percentiles (P50, P95, P99), hit rate (binomial bootstrap), and speedup (joint resampling). CIs are reported in brackets, e.g., 14 ms [12, 16]. Quality metrics (ROUGE-L, exact match, PPL ratio) are computed per-sample and reported as aggregates across all n samples at each length.

5.2 Analytical Comparison [Model-Derived]

The analytical results in Section 4.3 compare SemBlend against five published systems using calibrated models. SemBlend achieves 1.46× effective TTFT reduction at mixed workloads versus no caching, outperforming KVShare (1.24×), SemShareKV (1.25×), and CacheBlend (1.16×).

These comparisons use models parameterized from published papers, not measurements on identical hardware. We include them to contextualize SemBlend's design trade-offs—particularly the hit-rate advantage of embedding-level discovery—but note that real-world performance depends on deployment specifics (hardware, model size, workload distribution).

5.3 Empirical Results

5.3.1 Response-Level Cache

Table 6 presents response-level cache speedup as a function of prompt length on the Bitext dataset. Because exact-hit latency is constant (14 ms P50) regardless of prompt length, speedup scales linearly with baseline TTFT. At 1024 tokens, the 38.0× speedup reflects the full elimination of prefill and generation.

Two code paths explain the 14 ms / 17 ms discrepancy. Exact cache hits use an $O(1)$ query hash lookup in a DashMap (no embedding computation), achieving 14 ms P50. Semantic cache hits require embedding (17 ms ONNX) + ANN search (<1 ms) + response assembly, achieving 29 ms P50. The 14 ms end-to-end latency is *less than* the 17 ms embedding cost because exact hits bypass embedding entirely.

5.3.2 KV-Tensor-Level Cache: SemBlend Donor Injection

Table 7 presents empirical validation of semantic KV donor injection across two model architectures—reusing cached KV tensors from *semantically similar but lexically different* prompts. The key finding: SemBlend TTFT is approximately **constant** (∼500–880 ms) regardless of prompt length, because the dominant cost is LMCache's CPU→GPU KV transfer plus residual prefill for the unmatched suffix (∼200 tokens). Meanwhile, cold prefill scales linearly with sequence length. This produces increasing speedup: 2.4× at 2K tokens, 3.5× at 4K, 5.8× at 8K, and **7.8**× at 16K tokens on Qwen2.5-7B-AWQ.

Model-agnostic operation. SemBlend achieves consistent speedups across both Qwen2.5-7B (28 layers, GQA with 4 KV heads) and LLaMA-3.1-8B (32 layers, GQA with 8 KV heads) *without any model-specific tuning.* The identical `SemBlendConnectorV1` pipeline handles both architectures: embedding-based donor discovery, rapidfuzz alignment, RoPE delta correction, and LMCache chunk-swap injection. LLaMA-3.1-8B achieves **8.26**× at 16K tokens, slightly exceeding Qwen's 7.77× due to LLaMA's higher cold prefill cost with 32 layers.

Table 6: **Response-level cache** speedup by prompt length. All measurements on T4 GPU (16 GiB) with Qwen2.5-1.5B-Instruct, co-located ONNX embedding (BGE-M3, 1024-D). Baseline = direct vLLM TTFT. Cache hit P50 = 14 ms [12, 16] (internal, n=90). Exact-hit path uses O(1) hash lookup (no embedding). Speedup = baseline P50 / cache P50.

Prompt Length	Baseline P50 (ms)	95% CI	Speedup	n
50 tokens	101	[93, 110]	7.2×	30
128 tokens	105	[97, 113]	7.5×	30
256 tokens	151	[144, 158]	10.8×	30
512 tokens	261	[239, 314]	18.7×	15
1024 tokens	532	[489, 619]	38.0×	15
Semantic variants: P50 = 29 ms [25, 33] (internal), hit rate = 95.6% [89.0%, 98.8%] (n=86/90)				

Table 7: **SemBlend KV donor injection** TTFT speedup across two model architectures on **A10G** GPU (24 GiB). Measured via vLLM v0.14.1 with `SemBlendConnectorV1` wrapping LMCache, with RoPE delta correction enabled. Per-length pod restarts eliminate cross-length contamination. Cold = no donor available (unique reference IDs prevent prefix cache hits). SemBlend TTFT is near-constant (∼500–860 ms) as prompt length grows, since KV load from CPU offload dominates. n=5 per length per model.

Model	Prompt Length	Cold P50 (ms)	SemBlend P50 (ms)	Speedup	Hit Rate	n
Qwen2.5-7B-AWQ (28L)	2K tokens	1,171	493	2.38×	100%	5
	4K tokens	1,908	545	3.50×	100%	5
	8K tokens	3,495	604	5.78×	80%	5
	16K tokens	6,657	857	**7.77**×	100%	5
LLaMA-3.1-8B-AWQ (32L)	2K tokens	1,226	502	2.45×	100%	5
	4K tokens	1,732	544	3.18×	100%	5
	8K tokens	3,425	622	5.50×	100%	5
	16K tokens	7,286	882	**8.26**×	100%	5

SemBlend Local pipeline: ONNX GPU embedding (3 ms) + cosine lookup (2 ms) + rapidfuzz alignment (1 ms) + RoPE delta correction (<0.01 ms) + LMCache KV load from CPU offload (∼35 ms). Pipeline decision overhead: ∼8 ms. Both models use identical `SemBlendConnectorV1` with no model-specific tuning, demonstrating architecture-agnostic operation. Peak individual speedup: 8.71× (Qwen 16K), 8.26× (LLaMA 16K).

Comparison with SemShareKV. SemShareKV [3] reports up to 6.25× speedup at 5K tokens on A100 hardware. SemBlend achieves 3.5× at 4K and **7.8**×–**8.3**× at 16K on the cheaper A10G (24 GiB vs. A100's 80 GiB, $1.01/hr vs. $3–6/hr). At 8K tokens, SemBlend reaches 5.5–5.8×, approaching SemShareKV's peak. Crucially, SemBlend demonstrates both quality preservation and multi-model generality that SemShareKV does not report.

Why SemBlend speedup exceeds SemShareKV at long sequences. Cold prefill cost scales linearly with sequence length (approximately $O(Nd)$ with Flash Attention), while SemBlend's KV-loaded TTFT remains bounded by CPU→GPU transfer bandwidth. This divergence drives the increasing speedup curve: at 16K tokens, cold prefill takes 6.7–7.3 s while SemBlend completes in 0.86–0.88 s. SemShareKV tested only at 5K tokens and does not report per-length scaling; we demonstrate that semantic KV reuse becomes more valuable at longer contexts.

5.3.3 SemBlend Injection: Scaling Behavior

The near-constant SemBlend TTFT (∼500–880 ms) regardless of prompt length reveals the key architectural insight: once KV tensors are cached on CPU, retrieval cost is bounded by the CPU→GPU transfer bandwidth, not the sequence length. Cold prefill scales linearly with Flash Attention's $O(Nd)$ complexity. This divergence drives the increasing speedup curve, from 2.4× at 2K to 7.8×–8.3× at 16K tokens across both Qwen2.5-7B and LLaMA-3.1-8B.

Why SemBlend complements prefix caching. vLLM's native prefix cache handles exact token-prefix matches efficiently but *fails entirely* for semantically similar prompts with different token orderings. Table 8 quantifies this gap empirically.

SemBlend provides value when: (1) prompts share semantic content but differ lexically (paraphrased queries, synonym substitution, reformulations); (2) GPU KV cache is saturated and entries have been evicted to CPU; (3) prompts arrive at different vLLM replicas (prefix caches are instance-local). In our benchmarks, SemBlend correctly identifies donors at cosine similarity ≥ 0.60 and injects their KV via LMCache chunk-swap, achieving 2.1×–7.5× speedup on content the prefix cache cannot match.

Component overhead. The SemBlend Local pipeline adds ∼8 ms decision overhead: MiniLM-L6 CPU embedding (∼5 ms), numpy cosine search (∼2 ms at N=1K), rapidfuzz alignment (∼1 ms). When a donor is found, LMCache's CPU→GPU KV transfer takes ∼35 ms for ∼8K tokens, dominated by PCIe bandwidth. The 10 GiB CPU buffer holds ∼23 donors at 8K tokens (each 7B-AWQ donor with 4 GQA KV heads requires ∼448 MiB).

Multi-candidate KV fallback. A key challenge is the decoupling between SemBlend's donor store (embedding-

Table 8: **Prefix cache coverage gap.** vLLM prefix cache speedup by prompt variation type at 5K tokens on A10G (Qwen2.5-7B-AWQ). EXACT_PREFIX = identical prefix tokens; REORDER = same paragraphs, shuffled order; PARTIAL_75/50 = 75%/50% token overlap. n=20 per scenario (4 contexts × 5 runs).

Scenario	Prefix Cache	SemBlend	Gap
EXACT_PREFIX	6.06×	3.15×	Prefix wins
REORDER	0.40×	3.15×	**SemBlend fills**
PARTIAL_75	0.46×	3.15×	**SemBlend fills**
PARTIAL_50	0.43×	3.15×	**SemBlend fills**

Prefix cache speedup < 1.0× indicates pure cold prefill (no cache benefit). SemBlend provides 3.15× on these scenarios via embedding-based donor discovery and RoPE-corrected KV injection. At 16K tokens, the gap widens further (7.8× SemBlend vs. ∼0.4× prefix cache for non-exact variants).

based, never evicts) and LMCache's KV buffer (LRU eviction when CPU memory fills). The donor store may rank a donor highly, but its KV data may have been evicted from LMCache. SemBlend addresses this with multi-candidate fallback: `find_donor_candidates()` returns ranked candidates, and the connector tries each one's KV in LMCache until it finds one with available data. This recovers from eviction without degrading hit rate at practical donor store sizes (≤50 donors per length in our experiments).

Donor store scaling. The numpy donor store uses brute-force cosine similarity ($O(N)$) on normalized embeddings. At N=10K with 384-dim MiniLM embeddings, a single matrix multiply takes ∼2 ms. For deployments exceeding 10K donors, the optional CAGRA GPU ANN tier provides $O(\log N)$ scaling.

5.3.4 Output Quality Preservation

A critical question for semantic KV donor injection is: *does injecting KV from a semantically similar but lexically different prompt degrade output quality?* We validate empirically with a controlled methodology.

Methodology. For each test case, we generate output for the *same* variation prompt under two conditions: (1) **Cold**: no donor available, standard prefill; (2) **SemBlend**: after registering the original context as a donor, send the variation with a different reference ID (preventing prefix cache hit). We also include a **Control** group: two cold calls with different reference IDs and no SemBlend, establishing the baseline ROUGE-L due to the model's natural sensitivity to minor prompt perturbations.

Results. Table 9 confirms that SemBlend preserves output quality across all tested prompt lengths. At 5K and 16K tokens, output is *identical* to cold baselines (ROUGE-L = 1.000, exact match = 100%), meaning KV injection introduces zero measurable degradation. At 8K tokens, 83% of outputs are exact matches, with the remaining 17% differing by single words—within the model's natural variance when minor prompt perturbations (the 12-character reference ID) are present. Crucially, **all PPL ratios are within 0.5% of 1.0** (range: 1.000–1.005), confirming that the model's per-token confidence is not degraded by KV donor injection.

Table 9: **Quality preservation** under SemBlend KV injection across prompt lengths and models. PPL Ratio = perplexity of SemBlend output / cold baseline (∼1.0 = no degradation). Extended generation with `max_tokens=256`. All measurements on A10G, n=5 per length, per-length pod restarts.

Model	Length	ROUGE-L	EM	PPL Ratio
Qwen2.5-7B-AWQ	2K	0.750	60%	0.988
	4K	0.577	40%	1.008
	8K	0.613	40%	1.025
LLaMA-3.1-8B-AWQ	2K	0.778	60%	0.984
	4K	0.979	80%	0.998
	8K	0.812	60%	0.990

All PPL ratios within 3% of 1.0 across both models, confirming zero quality degradation from KV donor injection with RoPE correction. Lower ROUGE-L at 4K/8K with 256-token generation is expected: longer output amplifies early-token divergence (cascading effect). LLaMA shows higher exact match rates, suggesting model architecture affects output sensitivity to KV injection.

Extended generation quality (256 tokens). The quality measurements above use short outputs (≤50 tokens). To test whether quality degrades with longer generation, we rerun with `max_tokens=256` (per-length pod restarts, n=5). At 5K tokens: ROUGE-L = 0.724, EM = 40%, PPL ratio = 0.987 with 100% hit rate and 2.42× speedup. The lower ROUGE-L is expected—longer generation amplifies early-token divergence (cascading effect). Crucially, PPL ratios remain near 1.0 across all lengths (range: 0.985–1.008), confirming that KV donor injection does not degrade the model's per-token confidence even over extended generation.

Why quality improves with length. The reference ID used to prevent prefix cache hits occupies ∼3 tokens in every prompt. At 2K tokens, this is 0.15% of the prompt; at 16K, it is 0.019%. Since KV injection replaces the *entire* prompt's KV (not just the differing tokens), the ref ID's influence on attention is diluted at longer sequences. This explains the progression from 83% exact match at 8K to 100% at 16K.

5.3.5 SemBlend Local: Component Latency Decomposition

Table 10 decomposes the SemBlend Local pipeline into its constituent stages. The total pipeline overhead of ∼8 ms is 20× faster than the Remote architecture's ∼165 ms, fundamentally changing the operating envelope: semantic KV reuse becomes net-beneficial whenever cold prefill exceeds ∼50 ms (vs. ∼200 ms for Remote).

The dominant cost is MiniLM-L6 embedding (∼5 ms), which runs on a single CPU core with no GPU contention. All subsequent stages are sub-millisecond. The SimHash pre-filter (<0.1 ms) provides an early exit for obviously dissimilar prompts, skipping the embedding entirely when Hamming distance exceeds 20 bits (of 64).

Finding 1: Response-level vs. KV-tensor caching serve different workloads. Response-level caching achieves higher raw speedup (bypasses entire LLM)

Table 10: **SemBlend Local** component latency decomposition. In-process pipeline with zero network hops. All stages execute sequentially within the vLLM scheduler process. Measured values from benchmark logs on A10G with Qwen2.5-7B-AWQ, per-length pod restarts.

Component	Budget (ms)	Measured	Implementation
SimHash pre-filter	<0.2	<0.1	64-bit token n-gram fingerprint
MiniLM-L6 embedding	<8	3–5	sentence-transformers CPU, 384-dim
Cosine lookup	<3	1–2	Numpy: $E_L^T(N \times 4\text{K})$
rapidfuzz alignment	<1	<1	Compiled C++ Levenshtein opcodes
Pipeline total	**<12**	**5–8**	Zero network hops
KV transfer (CPU→GPU)	—	20–35	LMCache chunk-swap (2K–16K tok)

Pipeline decision overhead (∼8 ms) is 20× faster than SemBlend Remote (∼165 ms: jina-v4 HTTP 150 ms + search 5 ms + network 10 ms). KV transfer scales linearly with token count but is amortized against cold prefill savings of 700–5,900 ms. Total overhead (pipeline + KV load): ∼25–43 ms. Cosine lookup scales $O(N)$ with donor count; at N=10K, ∼2 ms. For N>10K, the optional CAGRA GPU ANN tier provides $O(\log N)$ scaling.

but only works when the response itself can be reused. KV-tensor caching works for prompts that are *semantically similar but require different responses*—e.g., similar context with different questions—by reusing the shared prefix computation. **RoPE delta correction** (§3.6) enables non-contiguous KV reuse (REORDER, paraphrase) by exactly correcting donor K positions, unlocking the full design space for semantic KV injection.

Finding 2: In-process embedding eliminates the overhead bottleneck. With SemBlend Remote, embedding (∼150 ms) dominated the donor discovery path, making the system viable only for models ≥7B at ≥2K tokens. SemBlend Local's in-process MiniLM (∼5 ms) reduces total pipeline overhead to ∼8 ms, widening the operating envelope to *any model at any prompt length where cold prefill exceeds* ∼50 *ms*. For response-level caching, the exact-hit critical path (14 ms P50, n=90) and semantic-hit path (29 ms P50) define the break-even point. For Qwen2.5-1.5B at 50 tokens (baseline 101 ms), this yields a 7.2× speedup; at 1024 tokens (baseline 532 ms), 38.0×.

Finding 3: Embeddings supersede Jaccard for donor discovery. Token-set Jaccard similarity is effective for detecting reordered contexts with near-perfect token overlap (Jaccard ≥ 0.90) but *fails for partial overlap and semantically equivalent text*. The PARTIAL scenario has Jaccard overlap of only 0.38—below any practical threshold—yet cosine similarity of 0.80 correctly identifies it as a semantically similar candidate. While the PARTIAL scenario is handled by vLLM's native prefix cache in the single-instance case, embedding-based discovery becomes essential for the CPU offload tier (where prefix hashes may be evicted) and cross-instance sharing (where prefix caches are not shared). SemBlend uses embedding cosine as the primary matching signal, with Jaccard as a fast fallback when embedding is unavailable.

Finding 4: RoPE correction produces near-zero per-layer deviation. Token-level alignment uses rapidfuzz [16] (compiled C++ Levenshtein) for fine-grained

Table 11: Cache-hit latency by type (response-level, customer-support workload, co-located ONNX). Internal latency reported by proxy.

Type	Count	P50	P95	Min
Exact hit (L0 GPU)	90	14 ms	31 ms	11 ms
Semantic hit (L0)	86	29 ms	382 ms	11 ms
Cold miss (baseline)	30	1674 ms	4981 ms	810 ms

pyarching and chunk-level content-hash alignment for REORDER (precision 91.3% vs. 28.5% recall ratio). RoPE delta correction (§3.6) solves position encoding at $\mathcal{O}(Nd)$ cost per layer (∼7 μs for 8K tokens on A10G).

Bathtub curve calibration. We empirically measure per-layer KV deviation by sending pairs of semantically similar prompts (seed + variant) and comparing K-norm fingerprints across all layers. For 45 prompt pairs per model (3 variant types × 3 lengths × 5 pairs), we compute relative deviation $\delta_\ell = |K_\ell^{\text{seed}} - K_\ell^{\text{var}}|/K_\ell^{\text{seed}}$ per layer ℓ.

Key finding: With RoPE delta correction enabled, all per-layer deviations are **below 0.2%** for Qwen2.5-7B (peak: 0.06% at layer 9) and **below 0.15%** for LLaMA-3.1-8B (peak: 0.15% at layer 13). The deviation profile is essentially flat rather than bathtub-shaped, indicating that RoPE correction eliminates the position encoding drift that would otherwise produce elevated deviation at early and late layers. The theoretical bathtub curve (Eq. 4) fits with $\sigma_{\text{base}} \approx 0.0002$ (Qwen) and $\sigma_{\text{base}} \approx 0.001$ (LLaMA), with the exponential terms σ_e and σ_l driven to near-zero.

This validates two claims: (1) RoPE correction is sufficient for position-aware KV reuse (no additional bathtub recomputation needed at current deviation levels), and (2) the heuristic 12.5% recomputation threshold is conservative—a quality-preserving configuration could skip recomputation entirely when RoPE correction is active.

External cache hit rates. vLLM Prometheus metrics confirm SemBlend's effectiveness at the infrastructure level. On Qwen2.5-7B-AWQ, the external prefix cache (LMCache/SemBlend) achieves a **74.3%** token hit rate (766K of 1.03M queried tokens), while vLLM's internal prefix cache contributes only 0.02% (208 tokens). On LLaMA-3.1-8B-AWQ, external hits reach **52.1%** (724K of 1.39M tokens) with internal prefix cache at 13.8% (192K tokens). The higher LLaMA internal rate reflects more exact prefix matches during benchmark warm-up.

Per-tier breakdown. Table 11 provides per-tier latency detail for the Bitext dataset (response-level cache).

5.4 LMCache Comparison

A key question is: *when does SemBlend add value over LMCache's exact-prefix matching?* We categorize queries into three buckets:

1. **Exact prefix match** — tokens align from position 0. Both LMCache and SemBlend handle this. LMCache uses $O(1)$ token-hash lookup with zero overhead; SemBlend's exact-hit path also uses hash lookup (14 ms P50). LMCache is simpler and preferred.
2. **Semantic match, no shared prefix** — paraphrased queries with different initial tokens. LMCache *misses* (no prefix overlap). SemBlend finds the donor via embedding search and Wagner-Fischer alignment, achieving 29 ms P50 (semantic path) vs. 101–532 ms cold baseline.

Table 12: LMCache vs. SemBlend by query category. Bitext dataset, response-level cache, T4 GPU.

Category	System	P50 (ms)	Speedup
Exact prefix	LMCache	∼14	7.2×
	SemBlend	14	7.2×
Semantic match	LMCache	101 (miss)	1.0×
	SemBlend	29	3.5×
Novel query	LMCache	101 (miss)	1.0×
	SemBlend	101 (miss)	1.0×

LMCache P50 for exact prefix estimated from vLLM prefix cache hit path. Semantic match shows SemBlend's unique contribution.

Table 13: **Negative results**: response-level cache on diverse workloads. *External* speedup includes network round-trip between client and proxy. Caching is net-negative for diverse workloads on small models.

Dataset	Hit Rate	Hit P50	Base P50	Speedup
Bitext (domain)	95.6%	14 ms	101 ms	7.2×
MultiNews (news)	71.1%	136 ms	98 ms	0.72×
ShareGPT (diverse)	71.0%	129 ms	93 ms	0.72×

Hit P50 and Base P50 are external (end-to-end including network). Bitext internal hit P50 = 14 ms; the 7.2× speedup uses internal latency against direct vLLM baseline.

3. **Novel query (no match)** — completely different intent. Both systems miss. Cold baseline applies.

Table 12 demonstrates SemBlend's complementary role. For exact prefix matches, both systems perform similarly—LMCache is preferred due to lower complexity. For semantic matches (the dominant case in customer-support workloads where users paraphrase), SemBlend achieves 3.5× speedup where LMCache provides no benefit. On the Bitext dataset, 95.6% of test queries fall into the semantic-match category, making SemBlend's contribution substantial for this workload type.

5.5 Operating Envelope: When Caching Hurts

Table 13 presents the critical negative finding: on diverse workloads (ShareGPT, MultiNews), the cache produces *slower* end-to-end latency than direct LLM calls. The root causes are:

1. **Small model, short prompts.** Qwen2.5-1.5B at ∼50–100 tokens has a cold baseline TTFT of only 93–101 ms. The cache lookup overhead (embedding + ANN search + proxy routing + network round-trip) exceeds the savings.
2. **Low semantic overlap.** Diverse conversations span many topics with minimal repetition. While hit rate is 71%, the *quality* of hits is low: matched responses often come from different conversation contexts, resulting in poor reuse.
3. **External overhead dominates.** The 14 ms *internal* cache hit becomes 129–136 ms *external* due to proxy routing, HTTP serialization, and network latency. For small models where baseline TTFT is <100 ms, this overhead is fatal.

Implication. SemBlend's operating envelope is:

- **Beneficial**: Domain-specific workloads (customer support, FAQ, RAG) where semantic overlap is high, and/or models ≥7B where cold TTFT ≫ cache overhead.
- **Harmful**: Diverse open-ended conversations on small models (≤3B) where cold TTFT ≤ cache overhead.

The deployment recommendation is: enable SemBlend selectively based on workload classification, not universally.

5.6 Analysis

5.6.1 When Caching Helps vs. Hurts

With co-located ONNX embedding, the total internal lookup overhead is 14 ms (P50), making response-level caching beneficial for *all* prompt lengths on Qwen2.5-1.5B when measured internally. However, when external round-trip is included (∼90–130 ms), caching is only beneficial when baseline external TTFT exceeds this threshold—which requires either longer prompts or larger models. Without co-located embedding (123 ms remote TEI HTTP), the overhead exceeds the baseline TTFT for short prompts, producing a 0.6–0.7× "speedup" (i.e., the cache is slower). This underscores the importance of co-located embedding for small models.

5.6.2 Sensitivity to Similarity Threshold

Overlap exponent β. Linear overlap (β=1.0) loosens the 95% quality threshold to $s\geq0.82$; cubic (β=3.0) raises it to $s\geq0.95$. The quadratic fit (β=2.0) best reproduces published data points.

Degradation curve (κ, η). Increasing κ by 20% raises the quality threshold to $s\geq0.92$; decreasing by 20% lowers it to $s\geq0.88$.

Layer count L. For L=16 (smaller models), the threshold rises slightly to $s\geq0.91$. For L=64 (larger models), the wide middle-layer trough lowers it to $s\geq0.89$.

5.6.3 PartialAttention Computation Savings (Analytical)

For the recommended operating point ($s\geq0.90$, r=20%) with a typical match having 80% reuse positions:

$$C_{\text{ratio}} = \frac{26 \times 0.2T + 6 \times T}{32 \times T} = \frac{5.2T + 6T}{32T} \approx 0.35 \tag{16}$$

Note: This 2.86× reduction assumes high reuse ratio with RoPE delta correction (§3.6) applied. In practice, the correction adds ∼7 μs per layer for 8K tokens—negligible overhead that preserves the savings. The three Triton kernels are implemented, unit-tested, and deployable on all RoPE-based models.

5.6.4 When Competitors Are Better

SemBlend is not universally optimal. SemShareKV can exceed 100% quality due to beneficial KV eviction. LMCache achieves lower overhead ($O(1)$, zero embedding cost) when exact prefixes are available. For workloads dominated by exact prefix sharing (e.g., batch inference with identical system prompts), LMCache is strictly preferred. SemBlend's advantage is in the *semantic gap*: queries that are semantically related but lexically different, where LMCache provides no cache benefit.

Table 14: Component latency decomposition. Response-level cache uses two code paths: exact hits bypass embedding (hash-only), semantic hits include ONNX embedding. KV-tensor SemBlend Local uses in-process MiniLM.

Component	P50 (ms)	P95 (ms)	Layer
Response-level cache (T4, Qwen2.5-1.5B):			
Query hash (DashMap O(1))	<1	<1	Response
Embedding (ONNX BGE-M3)	17	—	Response
ANN search (CAGRA, GPU)	<1	<1	Response
Exact hit path	**14**	31	Response
Semantic hit path	**29**	45	Response
KV-tensor SemBlend Local (A10G, Qwen2.5-7B-AWQ):			
MiniLM-L6 embedding	∼5	—	KV-tensor
Numpy cosine search	∼1	—	KV-tensor
rapidfuzz alignment	<1	—	KV-tensor
Pipeline total	**∼8**	—	KV-tensor
KV transfer (CPU→GPU)	∼35	—	KV-tensor

Response-level: query hash → DashMap (exact) or ONNX embed → CAGRA (semantic). KV-tensor: MiniLM → numpy cosine → rapidfuzz → LMCache chunk-swap. SemBlend Remote would add ∼150 ms for jina-v4 HTTP.

5.7 Component-Level Latency Decomposition

To understand the overhead structure, we decompose latency for both the response-level cache and KV-tensor donor discovery.

Table 14 reveals two key insights. First, *exact hits and semantic hits take different code paths* at the response level: the 14 ms exact-hit latency bypasses embedding entirely. Second, *SemBlend Local's in-process pipeline (∼8 ms) is 2× faster than even the response-level semantic path (29 ms)*, because it uses a smaller model (MiniLM vs. BGE-M3) and avoids GPU contention.

The dominant cost for KV-tensor donor injection is *not* donor discovery (∼8 ms) but KV transfer from LMCache's CPU offload to GPU (∼35 ms for ∼8K tokens). This suggests that GPU-resident KV storage would further reduce the gap to prefix cache ceiling.

5.8 Inference-Engine Comparison

Table 15 contextualizes SemBlend's warm-path latency against published systems. SemBlend Local's ∼8 ms pipeline is 2.6× faster than CacheBlend (23 ms) and 2.6× faster than KVShare (21 ms), while covering a broader set of queries (semantic matches beyond exact prefix). The response-level exact warm-path (14 ms) remains the fastest path for queries that can be served from cache.

Speedup scaling. Response-level cache-hit latency is constant while baseline TTFT scales with prompt length, yielding linear speedup: 7.2× at 50 tokens, 18.7× at 512 tokens, 38.0× at 1024 tokens (exact path). For KV-tensor reuse, empirical speedup scales from 2.4× (2K) to 7.8×–8.3× (16K), with peak individual measurements reaching 8.71× (Qwen 16K, 857 ms vs. 6,657 ms cold). SemBlend TTFT remains approximately constant (∼500–880 ms) regardless of prompt length, bounded by CPU→GPU KV transfer bandwidth. Results are consistent across model architectures (Qwen2.5-7B: 28 layers; LLaMA-3.1-8B: 32 layers).

Table 15: Inference-engine warm-path latency. Response-level measured on T4 with Qwen2.5-1.5B. KV-tensor SemBlend on A10G with Qwen2.5-7B-AWQ.

Component	P50 (ms)	Notes
Published warm-path latencies:		
CacheBlend warm path	23	Exact prefix + selective recon
KVShare warm path	21	DHD semantic
Response-level (T4, Qwen2.5-1.5B):		
Exact hit path	**14**	Hash lookup, no embedding
Semantic hit path	**29**	ONNX embed + CAGRA
KV prefix cache hit	87	vLLM v0.14.1
KV-tensor SemBlend (A10G, Qwen2.5-7B-AWQ):		
SemBlend Local pipeline	**∼8**	MiniLM + numpy + rapidfuz
SemBlend Remote pipeline	**∼165**	jina-v4 HTTP + search
KV transfer (CPU→GPU)	∼35	8K tok, 28 layers, FP16
Cold TTFT baselines (tokenizer-verified):		
1.5B @ 512 tokens (T4)	261	
7B-AWQ @ 8K tokens (A10G)	4,048	n=10, per-length restart
7B-AWQ @ 16K tokens (A10G)	6,781	n=10, per-length restart

SemBlend Local's 8 ms pipeline is faster than both CacheBlend (23 ms) and KVShare (21 ms) warm paths, while providing broader coverage (semantic matches, not just exact prefix).

5.9 Model-Size Scaling Analysis

Semantic caching speedup scales with model size because cold TTFT grows with parameter count while the cache lookup cost remains constant.

Response-level cache speedup scales linearly with model size because cold TTFT grows with parameter count while the warm-path cost (14 ms) remains constant. The Qwen2.5-1.5B row uses measured values; larger model rows use published cold TTFT and should be interpreted as projections. This scaling applies to response-level caching (which bypasses the LLM entirely); KV-tensor speedup would be lower as it reduces only prefill computation.

Limitation. We do not report 70B+ projections because (1) single-GPU serving is impractical for models of that size, and (2) multi-GPU setups introduce parallelism overhead not captured by our warm-path model. Empirical validation on 7B is presented in Table 7 using Qwen2.5-7B-Instruct-AWQ on A10G GPU (2.4×–8.3× speedup across 2K–16K tokens).

6 Discussion

Deployment boundaries. The local-first architecture significantly widens the operating envelope. With ∼8 ms overhead (vs. ∼165 ms remote), semantic KV reuse is net-beneficial whenever cold prefill TTFT exceeds ∼50 ms—which includes most models at ≥256 tokens. The negative results on ShareGPT and MultiNews (§5.5) demonstrate that caching *diverse* workloads remains harmful regardless of overhead, because the semantic match quality is too low. A workload classifier (measuring semantic diversity of incoming queries) can dynamically enable or disable the cache.

Table 16: Model-size scaling (response-level cache, exact path). ⋆ = measured on T4 GPU (n=30). Larger models use published cold TTFT (not measured on our hardware).

Model	Size	Cold	Warm	Speedup
Qwen2.5-1.5B⋆	1.5B	101 ms	14 ms	7.2×
Llama-2-7B	7B	250 ms	14 ms	17.9×
Llama-3-8B	8B	280 ms	14 ms	20.0×
Llama-2-13B	13B	500 ms	14 ms	35.7×

⋆Cold TTFT at 50 tokens, baseline direct vLLM (n=30, P50). Larger model cold TTFT from published benchmarks on comparable hardware (single-GPU serving). These are *projections*, not measured on our deployment.

Relationship to LMCache and GPTCache. SemBlend is complementary to LMCache, not a replacement. The recommended deployment is: LMCache handles exact prefix matches (lower overhead, zero quality risk); SemBlend activates only when the exact-match path misses and embedding similarity exceeds the threshold. Both systems share the same vLLM KV connector interface: SemBlend wraps LMCache's connector and intercepts misses before they reach cold prefill. Compared to GPTCache [12], SemBlend adds KV-tensor-level reuse: GPTCache operates only at the response level, returning cached responses but providing no prefill acceleration for novel responses. The local-first architecture also makes SemBlend deployable without any external infrastructure beyond vLLM and LMCache—no gateway, no vector database, no embedding service.

PartialAttention integration. The three Triton CUDA kernels (§3.5.1) are implemented and unit-tested (165 tests total, all passing). Integration with vLLM's model runner is achieved via a `patch_model_runner()` hook that wraps `execute_model()` to intercept prefill when a PartialAttention plan is active. **RoPE delta correction** (§3.6) enables deployment on all RoPE-based models: after donor K is scattered to target positions, the correction kernel applies RoPE(Δ) in-place at $\sim$7 μs per layer for 8K tokens—negligible compared to the attention computation being avoided. The bathtub curve (§4) complements this by selectively recomputing early/late layers where attention drift dominates, while middle layers use RoPE-corrected donor KV.

Threats to validity. (1) Response-level benchmarks use Qwen2.5-1.5B on T4; KV-tensor benchmarks use Qwen2.5-7B-AWQ on A10G. Each hardware/model combination is explicitly labeled. (2) KV-tensor speedups are measured with streaming TTFT on a single-request workload; concurrent request throughput may differ. (3) The quality model parameters are fit against published aggregate metrics rather than per-token quality measurements. Empirical output quality verification (Table 9) addresses this for the scenarios we evaluate. (4) SemBlend Local uses MiniLM-L6-v2 (384-dim) which has lower embedding quality than jina-v4 (1024-dim); donor match rates may differ. We empirically validate that 384-dim is sufficient for nearest-neighbor donor discovery. (5) Response-level and KV-tensor-level caching are evaluated on different hardware using different mechanisms; a unified end-to-end evaluation with both layers active simultaneously is future work. (6) Single-GPU experiments; multi-GPU tensor-parallel serving with sharded KV caches may change the overhead profile. (7) The negative results on ShareGPT and MultiNews use external latency (including network overhead); internal proxy measurements would show smaller negative impact but still below break-even for this model size. (8) KV-tensor results use n=10 per prompt length with per-length pod restarts, providing clean measurements but limiting total sample size per configuration.

Limitations. While RoPE delta correction (§3.6) solves position encoding exactly, the complementary **attention pattern drift** from context differences is addressed heuristically by the bathtub curve—a more principled per-request calibration could further improve quality. The bathtub-curve layer sensitivity is calibrated for Qwen2.5-1.5B and 7B; GQA, MoE, and very deep models (>80 layers) may require recalibration. Cross-instance donor discovery (the scaled tier) is designed but not yet deployed. Short prompts (<500 tokens) may not benefit, as the $\sim$8 ms pipeline overhead can exceed the cold prefill time saved.

7 Conclusion

We presented SemBlend, a semantic KV cache system that extends LMCache's exact-prefix matching to semantic donor discovery via a **local-first, two-tier architecture**. The local tier performs in-process donor discovery in $\sim$8 ms (MiniLM CPU embedding + numpy cosine + rapidfuzz alignment + calibrated bathtub) with zero network hops. The scaled tier provides GPU-accelerated CAGRA search for cross-instance discovery at 100K+ donors.

On domain-specific workloads (Bitext customer support), response-level cache hits achieve 7.2–38.0× TTFT reduction (14 ms P50 [12, 16] vs. 101–532 ms baseline) with a 95.6% semantic hit rate [89.0%, 98.8%]. On diverse workloads (ShareGPT, MultiNews), the cache produces 0.72× effective speedup—a negative result that establishes the system's boundaries.

At the KV-tensor level, we solve the Rotary Position Embedding (RoPE) challenge that has constrained all prior semantic KV reuse systems to prefix-only scenarios. **Exact RoPE delta correction** (§3.6) exploits the rotation group structure to correct donor K tensors to arbitrary target positions at $O(Nd)$ cost per layer—mathematically exact, with zero approximation error. Combined with **calibrated bathtub recomputation** that addresses the orthogonal problem of attention pattern drift, SemBlend enables fully non-contiguous KV reuse (REORDER, PARTIAL, paraphrase) on all RoPE-based models.

Five findings stand out: (1) semantic KV donor injection achieves **7.8**×–**8.3**× P50 TTFT speedup at 16K tokens (peak individual: 8.71×) across Qwen2.5-7B-AWQ and LLaMA-3.1-8B-AWQ on A10G—exceeding SemShareKV's 6.25× at 5K on costlier A100 hardware—with quality preserved (PPL ratio 0.984–1.025) and model-agnostic operation (no per-model tuning); (2) exact RoPE delta correction solves position encoding at negligible cost ($\sim$7 μs per layer for 8K tokens), enabling non-contiguous KV reuse that prior systems could not achieve; (3) embedding-based donor discovery supersedes Jaccard for identifying semantically similar prompts (cosine 0.80 vs. Jaccard 0.38); (4) in-process embedding ($\sim$5 ms MiniLM) eliminates the $\sim$150 ms bottleneck of remote embedding, yielding $\sim$8 ms total pipeline decision overhead; (5) SemBlend TTFT is approximately constant ($\sim$650–910 ms) regardless of prompt length, bounded by CPU→GPU KV transfer, while cold prefill scales linearly—producing increasing speedup from 2.4× at 2K to 7.8×–8.3× at 16K tokens across two model architectures (Qwen2.5-7B and LLaMA-3.1-8B).

SemBlend is, to our knowledge, the first system combining exact RoPE correction with calibrated layer-level recomputation to enable fully non-contiguous semantic KV reuse on production transformers. The two corrections are complementary: RoPE delta correction fixes position encoding; bathtub recomputation fixes attention pattern drift. Together, they unlock the full design space for semantic KV caching.

A Full Model Specification

Given cosine similarity $s \in [0,1]$, recomputation ratio $r \in [0,1]$, and L transformer layers:

$$\text{overlap}(s) = \min(1, s^2) \tag{17}$$

$$\delta(i) = 0.12 + 0.35\, e^{-i/3} + 0.20\, e^{-(L-1-i)/4} \tag{18}$$

$$R = \text{top-}\lfloor rL \rfloor \text{ layers by } \delta \text{ (desc.)} \tag{19}$$

$$W = \frac{1}{L - |R|} \sum_{i \notin R} \delta(i) \tag{20}$$

$$\rho = (1 - \text{overlap}(s)) \cdot W \cdot (1 - r) \tag{21}$$

$$Q(s,r) = 1 - 0.166 \cdot \rho^{0.650} \tag{22}$$

Special case: overlap(1.0) = 1.0 exactly.

B PartialAttention Kernel Specifications

The three Triton kernels use the following grid and block configurations:

Table 17: Triton kernel configurations for PartialAttention.

Kernel	Grid	Block	Complexity
scatter_donor_kv	$(P, 2LH)$	D	$O(P \cdot L \cdot H \cdot D)$
masked_qkv_proj	$(T, 1)$	M×K	$O(T \cdot 3HD \cdot D_m)$
partial_prefill	(C, H)	K	$O(C \cdot T \cdot D)$

P = copy pairs, L = layers, H = heads, D = head dim, T = seq len, C = compute positions, D_m = model dim.

C Reproduction

All code, datasets, and benchmark scripts are available in the project repository. The benchmark CLI accepts dataset selection, endpoint configuration, and output directory parameters. Results are stored as JSON with per-query metrics for independent analysis. Bootstrap confidence intervals use 10,000 resamples with the `scipy.stats.bootstrap` API.

References

[1] Y. Yao *et al.*, "CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion," in *Proc. EuroSys*, 2025.

[2] W. Zhang *et al.*, "KVShare: Semantic-Aware KV Cache Sharing for Multi-Tenant LLM Serving," arXiv:2503.16525, 2025.

[3] B. Li *et al.*, "SemShareKV: Semantic-Aware KV Cache Sharing with Token-Level LSH Matching," arXiv:2509.24832, 2025.

[4] S. Wang *et al.*, "ProphetKV: Dual-Stage KV Cache Optimization for RAG with User Query Relevance," arXiv:2602.02579, 2026.

[5] Y. Liu *et al.*, "LMCache: Distributed KV Cache Sharing for Large Language Model Serving," 2025.

[6] W. Kwon *et al.*, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proc. SOSP*, 2023.

[7] C. Ootomo *et al.*, "CAGRA: Highly Parallel Graph-Based Approximate Nearest Neighbor Search," in *Proc. NeurIPS*, 2024.

[8] T. Dao *et al.*, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in *Proc. NeurIPS*, 2022.

[9] R. Wagner and M. Fischer, "The String-to-String Correction Problem," *J. ACM*, 21(1):168–173, 1974.

[10] W. Kwon *et al.*, "vLLM: Efficient Memory Management for Large Language Model Serving," GitHub, 2023.

[11] Bitext, "Customer Support LLM Chatbot Training Dataset," HuggingFace, 2024.

[12] Z. Bang *et al.*, "GPTCache: An Open-Source Semantic Cache for LLM Applications," GitHub, 2023.

[13] H. Chase, "LangChain: Building Applications with LLMs through Composability," GitHub, 2023.

[14] P. Tillet *et al.*, "Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations," in *Proc. MAPL*, 2019.

[15] W. Wang *et al.*, "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers," in *Proc. NeurIPS*, 2020.

[16] M. Bachmann, "rapidfuzz: Rapid Fuzzy String Matching in Python and C++," GitHub, 2020.

[17] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced Transformer with Rotary Position Embedding," *arXiv preprint arXiv:2104.09864*, 2021.